

P1144R13

std::is_trivially_relocatable

Published Proposal, 2025-05-13

Authors:

[Arthur O'Dwyer](#)

[Artur Bać](#)

[Daniel Liam Anderson](#)

[Enrico Mauro](#)

[Jody Hagins](#)

[Michael Steffens](#)

[Stéphane Janel](#)

[Vinnie Falco](#)

[Walter E. Brown](#)

[Will Wray](#)

Audience:

WG21

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

P1144-style trivial relocation is used by Abseil, AMC, BSL, Folly, HPX, Parlay, Pocketpy, Qt, Subspace, and Thrust. Let's make it part of the C++ standard.

Table of Contents

- 1 What's going on here?**
 - 1.1 Statement from Stéphane Janel
 - 1.2 Statement from Vinnie Falco
 - 1.3 Statement from Michael Steffens

1.4 Statement from Arthur O'Dwyer

2 Scope and design

3 Proposed wording

- 3.1 [cpp.predefined]
- 3.2 [version.syn]
- 3.3 [basic.types.general]
- 3.4 [class.pre]
- 3.5 [class.prop]
- 3.6 [cpp.cond]
- 3.7 [dcl.attr.trivreloc]
- 3.8 [expr.prim.lambda.closure]
- 3.9 [lex.name]
- 3.10 [library.class.props]
- 3.11 [meta.type.synop]
- 3.12 [meta.unary.prop]
- 3.13 [obj.lifetime]
- 3.14 [diff.cpp23.dcl.dcl]

References

Informative References

§ 1. What's going on here?

Some of us submitted [P3236] for St Louis 2024, indicating our disapproval of [P2786R4] and asking for our preferred direction [P1144R10] to be discussed in committee. This resulted in EWG's "clawing back" P2786 for one meeting, but did not result in any committee discussion of P1144, due to its author.

[P2786R13] was then merged into the Working Draft ([N5008]) at Hagenberg 2024. Almost all of P3236's objections remain unaddressed. Here are P3236's objections again:

- "Types with user-provided `operator=` are reported as trivially relocatable by default." Addressed.
- "The programmer is allowed to create a type that is trivially copyable but not trivially relocatable." Not addressed.

- "It's ill-formed to create a trivially relocatable type with an `offset_ptr` member, or even a Boost `static_vector` member." Not addressed.
- "No backward-compatible syntax. P2786 proposes a new keyword, and goes out of its way to make the keyword unusable in C++23-and-earlier." Not addressed.
- "P1144, by providing stronger guarantees of value semantics, permits more optimizations." Not addressed.
- "P1144 provides a well-designed and stable library API." Addressed, in that the P1144 library API has been approved for C++26 via [P3516].
- "P2786's warrant marking is 'viral downward.' As container authors, we think P2786's normalization of a large number of explicit markings will cause programmer fatigue and lead to bugs." Not addressed.
- "Deliberately incomplete. P2786's authors have drafted followup papers including [P2959] and [P2967], all with incomplete wording and 'heroic' changes, for example `std::container_replace_with_assignment`." Arguably addressed, in that P2786 was merged in its present state, and P2959 and P2967 seem to have been abandoned. We now know with certainty what we have gotten, and we can say with certainty that it does not suit us.

In addition, the Working Draft version of P2786R13 contains several problems that weren't present in P2786R4 at the time we wrote [P3236].

- The ARM64e platform uses "vp_{tr} signing." Its vp_{tr}s are never safe to memcpy. Yet the Working Draft permits `is_trivially_relocatable` to return `true` for types with vp_{tr}s, types recursively containing members with vp_{tr}s, etc. This means that it is never safe to use e.g. `realloc` on ARM64e even when we know that the objects being reallocated are all `is_trivially_relocatable`. New problem, not addressed.
- P2786's approved syntax is shockingly ugly: `class C trivially_relocatable_if_eligible replaceable_if_eligible { }`. P1144's attribute syntax is unobtrusive by comparison: `class [[trivially_relocatable]] C { }`. New problem, not addressed.

Therefore we submit this revision of P1144, which addresses all of these problems directly, as a diff against the current Working Draft. We gladly list ourselves as coauthors of this proposal, in the same way that five Bloomberg employees listed themselves as coauthors of [P2786R13]. Unlike them, we come from a variety of backgrounds and maintain a variety of codebases.

We beg WG21 to resolve the community's concerns before it is too late.

§ 1.1. Statement from Stéphane Janel

Stéphane Janel is the maintainer of [AmadeusITGroup/amc](#), a collection of high-performance C++ containers used in the Amadeus pricing and shopping engines.

Hello Arthur,

As already discussed together, I am hugely in favor of P1144 against P2786.

The reasons are perfectly described [here](#) and for me the main point is that P1144 better integrates with current type traits, and is much more natural to use. I don't know how to target WG21 though, feel free to forward my email if it can help.

§ 1.2. Statement from Vinnie Falco

Vinnie Falco is the maintainer of [Boost.JSON](#), a portable C++ library of containers and algorithms that implement the JSON lightweight data-interchange format.

Boost.JSON makes intense use of trivial relocation for its variant-like container, [boost::json::value](#), which can hold any kind of JSON node including arrays, objects, and strings.

This means when resizing an array of values (represented by the type `boost::json::array`) or an unordered container of values (represented by the type `boost::json::object`), the implementation can [simply relocate](#) the contents of the container. The implementation of `relocate()` for an array of value is [simply memcpy](#).

Since Boost.JSON requires only C++11 we have to make some assumptions which are in principle not portable, yet in practice work everywhere (we have not gotten any reports of malfunctions). To keep the library's data types clean and first-class, we adopted Peter Dimov's "pilfer construction" approach from [P0308R0](#), which is related to trivial relocation.

I'm happy to answer questions about the library if it can help the committee make better design choices, although it seems like decisions have already been made. Unfortunately I do not have the motivation to get more involved as there is little indication that quality of outcome is proportional to the level of effort invested.

§ 1.3. Statement from Michael Steffens

Dr. Michael Steffens is the author of [misteffens/TrivialityInversion](#), a public proof-of-concept for storage-location-agnostic C++ container types. His `BoundedVector` type includes an `offset_ptr-` into-self as a data member, making `BoundedVector` trivially relocatable (for most types) even though one of its data members is not.

It seems that you are raising a lot of concerns with P2786 — on different abstraction levels — but the key concern in my context seems to be the lack of opt-in trivial relocatability. I have read arguments in favor of that opt-out-only, claiming that it's safer not to allow opt-in, overriding what the compiler could deduce on its own, as that could be done in error. And it's not seen as a functional gap, but only a missed opportunity for optimization.

In my particular case (as in `boost::interprocess`) the motivation is not optimization, but storing objects in shared memory and coping with their different address offsets in different processes. As C++ standards do not cover IPC at all yet, we map the use of SHM to trivial copying or relocation, as if objects would end their life in one process and be memmoved to the other process (and back, as needed), being aware that the memmove never actually happens. Instead it's kind of an excuse to start/end object lifetimes on the other side without construction or destruction. This use case cannot be covered at all without the opt-in.

If I understood everything right until this point, sure, I'd like to lend a hand on the P1144 side, based on the need to have a workaround for IPC.

§ 1.4. Statement from Arthur O'Dwyer

Arthur O'Dwyer is the maintainer of [Quuxplusone/SG14](#), a library of containers and algorithms pioneered by the low-latency study group. He also maintains a non-public `Vector` implementation and a public P1144+P3516-compliant [fork of libc++](#). He was previously the sole credited author of P1144 up to R12.

I want to be clear: I did not *invent* trivial relocation in my P1144R0. That paper was based on implementations already in the wild in 2018: notably BSL, Qt, and Folly. Since then, more than a dozen other libraries have adopted the same semantics and most (including Qt) have adopted the terminology from P1144 and P3516. At least five library authors have written blogs or documentation about their use of "trivial relocation": [Folly](#), [AMC](#), [Qt](#), [Subspace](#), [HPX](#). Without exception, the people using it and writing about it follow the same path: the semantics recorded in (not originated by!) P1144 and P3516.

Standardizing the prevailing term of art, as documented in the five links above, would be great. WG21's repurposing the prevailing term for a different meaning will lead only to confusion and safety bugs.

§ 2. Scope and design

In this rebased proposal, we propose:

- The set of trivially relocatable types should be exactly those types which can be relocated as-if by memcpy. We change the definition of "trivially relocatable class" ([\[basic.types.general\]](#)) to include all trivially copyable types, and to exclude polymorphic types which require additional fixup beyond a simple memcpy. We need trivial relocatability to imply memcpy-ability and realloc-ability, or it is useless to us.
- Remove the possibility that `std::trivially_relocate` ([\[obj.lifetime\]](#)) might do anything other than a simple memcpy of the underlying bytes. Otherwise we preserve the `std::trivially_relocate` and `std::relocate` functions; we do not mind them. What is important is that they must be *trivial*; they must not be allowed to perform non-trivial "fixup" of the relocated objects (e.g. to fix up the vptrs of polymorphic objects).
- Remove the terms *default-movable* and *eligible for trivial relocation* ([\[class.prop\]](#)). They are no longer needed.
- Remove the terms *replaceable type* ([\[basic.types.general\]](#)), *eligible for replacement*, and *replaceable class* ([\[class.prop\]](#)) and the `std::is_replaceable` type-trait ([\[meta.type.synop\]](#)). Nothing in [\[N5008\]](#) (nor [\[P3516\]](#)) depends on these notions. They are likely to cause confusion to end-users, particularly in their similarity to the existing term of art "transparently replaceable" ([\[basic.life\]](#)). We believe they [are not useful](#) to library authors.
- Remove the `trivially_relocatable_if_eligible` and `replaceable_if_eligible` keywords ([\[lex.name\]](#)). They are ugly. They are difficult to use correctly because they are "viral"; see [\[P3236\]](#). They are unusable outside of C++26 mode. We propose to replace both contextual

keywords with a single ignorable attribute, which compiler vendors can choose to support even in older C++ modes if they want.

- Remove the Annex C entry ([\[diff.cpp23.dcl.dcl\]](#)) and discussion of new grammatical ambiguities ([\[class.pre\]](#)) caused by the P2786 syntax. These grammatical issues no longer exist.
- Preserve the `std::is_trivially_relocatable` and `std::is_nothrow_relocatable` type-traits.
- Add the `[[trivially_relocatable]]` attribute as used by `std_error`, `parlaylib`, and `SG14`. Note the lack of "if eligible" qualification: this attribute has "sharp knife" semantics suitable for our purposes and matching our current use-cases. (See [\[P3236\]](#).) Unlike the contextual keywords, this attribute can be supported in older C++ modes too.
- We do not need to propose P1144's library functionality — `std::uninitialized_relocate`, `std::relocate_at`, and so on — because that functionality has been submitted in a separate paper not authored by Arthur O'Dwyer ([\[P3516\]](#)) which LEWG has approved for C++26. We are happy with this outcome.

§ 3. Proposed wording

The wording in this section is relative to the current Working Draft, [\[N5008\]](#).

§ 3.1. [\[cpp.predefined\]](#)

Modify the feature-test macros in the table in [\[cpp.predefined\]](#):

```
__cpp_impl_three_way_comparison    201907L
__cpp_impl_trivially_relocatable   YYYYMM
__cpp_implicit_move                202207L

[...]

__cpp_threadsafe_static_init       200806L
__cpp_trivial_relocatability       202502L
__cpp_trivial_union                202502L
```

§ 3.2. [\[version.syn\]](#)

Bump the feature-test macro in [\[version.syn\]/2](#):

```
#define __cpp_lib_transparent_operators 201510L // freestanding, also in <memory>
#define __cpp_lib_trivially_relocatable 202502L YYYYMM // freestanding, also in <memory>
#define __cpp_lib_tuple_element_t 201402L // freestanding, also in <tuple>
```

§ 3.3. [basic.types.general]

Add a new section in [basic.types.general]:

9. Arithmetic types ([basic.fundamental]), enumeration types, pointer types, pointer-to-member types ([basic.compound]), `std::nullptr_t`, and cv-qualified versions of these types are collectively called *scalar types*. Scalar types, trivially copyable class types ([class.prop]), arrays of such types, and cv-qualified versions of these types are collectively called *trivially copyable types*.

Scalar **Trivially copyable** types, trivially relocatable class types ([class.prop]), arrays of such types, and cv-qualified versions of these types are collectively called *trivially relocatable types*. ~~Cv-unqualified scalar types, replaceable class types ([class.prop]), and arrays of such types are collectively called *replaceable types*.~~ Scalar types, standard-layout class types ([class.prop]), arrays of such types, and cv-qualified versions of these types are collectively called *standard-layout types*. Scalar types, implicit-lifetime class types ([class.prop]), array types, and cv-qualified versions of these types are collectively called *implicit-lifetime types*.

[Note: For a trivially relocatable type, object relocation operations (as exemplified by `std::swap_ranges` and `std::vector::reserve`) are always tantamount to simple copies of the underlying bytes. —end note]

10. A type is a *literal type* if it is: [...]

§ 3.4. [class.pre]

Modify [class.pre] as follows:

class-property-specifier:

```
final  
trivially_relocatable_if_eligible  
replaceable_if_eligible
```

[...]

5. Each *class-property-specifier* shall appear at most once within a single *class-property-specifier-seq*. Whenever a *class-key* is followed by a *class-head-name*, the identifier `final`, ~~`trivially_relocatable_if_eligible`~~, or ~~`replaceable_if_eligible`~~, and a colon or left brace, the identifier is interpreted as a *class-property-specifier*.

[Example:

```
struct A;  
struct A final {};           // OK, definition of struct A,  
                             // not value-initialization of variable final  
  
struct X {  
    struct C { constexpr operator int() { return 5; } };  
    struct B trivially_relocatable_if_eligible final : C{};  
                                     // OK, definition of nested class B,  
                                     // not declaration of a bit-field member  
                                     // trivially_relocatable_if_eligible final  
};
```

—end example]

§ 3.5. [class.prop]

Modify [class.prop] as follows:

1. A *trivially copyable class* is a class:

- that has at least one eligible copy constructor, move constructor, copy assignment operator, or move assignment operator ([special], [class.copy.ctor], [class.copy.assign]),
- where each eligible copy constructor, move constructor, copy assignment operator, and move assignment operator is trivial, and
- that has a trivial, non-deleted destructor ([class.dtor]).

x. A *trivially relocatable class* is a class:

- where no eligible copy constructor, move constructor, copy assignment operator, move assignment operator, or destructor is user-provided,
- that has no virtual member functions or virtual base classes,
- all of whose non-static data members are either of reference type or of trivially relocatable type ([basic.types.general]), and
- all of whose base classes are of trivially relocatable type;

or a class that is declared with a `trivially_relocatable` attribute with value `true` ([dcl.attr.trivrelloc]) if that attribute is supported by the implementation ([cpp.cond]).

2. A class *C* is *default-movable* if

- overload resolution for direct-initializing an object of type *C* from an xvalue of type *C* selects a constructor that is a direct member of *C* and is neither user-provided nor deleted,
- overload resolution for assigning to an lvalue of type *C* from an xvalue of type *C* selects an assignment operator function that is a direct member of *C* and is neither user-provided nor deleted, and
- *C* has a destructor that is neither user-provided nor deleted.

3. A class is *eligible for trivial relocation* unless it

- has any virtual base classes,
- has a base class that is not a trivially relocatable class,
- has a non-static data member of an object type that is not of a trivially relocatable type, or
- has a deleted destructor,

except that it is implementation-defined whether an otherwise-eligible union having one or more subobjects of polymorphic class type is eligible for trivial relocation.

4. A class *C* is a *trivially relocatable class* if it is eligible for trivial relocation and

- has the `trivially_relocatable_if_eligible` class-property-specifier,
- is a union with no user-declared special member functions, or
- is default-movable.

5. [Note: A class with const-qualified or reference non-static data members can be trivially relocatable. —end note]

6. A class *C* is *eligible for replacement* unless

- it has a base class that is not a replaceable class;
- it has a non-static data member that is not of a replaceable type;
- overload resolution fails or selects a deleted constructor when direct-initializing an object of type *C* from an xvalue of type *C* ([*dcl.init.general*]);
- overload resolution fails or selects a deleted assignment operator function when assigning to an lvalue of type *C* from an xvalue of type *C* ([*expr.assign*], [*over.assign*]); or
- it has a deleted destructor.

7. A class *C* is a *replaceable class* if it is eligible for replacement and

- has the `replaceable_if_eligible_class_property_specifier`;
- is a union with no user-declared special member functions, or
- is default-movable.

8. [Note: Accessibility of the special member functions is not considered when establishing trivial relocatability or replaceability. —end note]

9. [Note: Not all trivially copyable classes are trivially relocatable or replaceable. —end note]

10. A class *S* is a *standard-layout class* if it: [...]

§ 3.6. [cpp.cond]

Add a new entry to the table of supported attributes in [cpp.cond]:

noreturn	200809L
<u>trivially_relocatable</u>	YYYYMML
unlikely	201803L

§ 3.7. [dcl.attr.triv reloc]

Add a new section after [dcl.attr.nouniqueaddr]:

1. The *attribute-token* `trivially relocatable` specifies that a class type's relocation operation has no visible side-effects other than a copy of the underlying bytes, as if by the library function `std::memcpy`. It may be applied to the definition of a class. It shall appear at most once in each *attribute-list*. An *attribute-argument-clause* may be present and, if present, shall have the form

(*constant-expression*)

The *constant-expression* shall be an integral constant expression of type `bool`. If no *attribute-argument-clause* is present, it has the same effect as an *attribute-argument-clause* of `(true)`.

2. If any definition of a class type has a `trivially relocatable` attribute with value *V*, then each definition of the same class type shall have a `trivially relocatable` attribute with value *V*. No diagnostic is required if definitions in different translation units have mismatched `trivially relocatable` attributes.

3. If a class type is declared with the `trivially relocatable` attribute, and the program relies on observable side-effects of its relocation other than a copy of the underlying bytes, the behavior is undefined.

4. *Recommended practice:* The value of a *has-attribute-expression* for the `trivially relocatable` attribute should be `0` for a given implementation unless this attribute can cause a class type to be trivially relocatable ([`class.prop`]).

§ 3.8. [`expr.prim.lambda.closure`]

Modify [`expr.prim.lambda.closure`] as follows:

3. The closure type is not an aggregate type ([`dcl.init.aggr`]); it is a structural type ([`temp.param`]) if and only if the lambda has no *lambda-capture*. An implementation may define the closure type differently from what is described below provided this does not alter the observable behavior of the program other than by changing:

- the size and/or alignment of the closure type,
- whether the closure type is trivially copyable ([`class.prop`]),
- whether the closure type is trivially relocatable ([`class.prop`]),
- ~~whether the closure type is replaceable ([`class.prop`])~~, or
- whether the closure type is a standard-layout class ([`class.prop`]).

An implementation shall not add members of rvalue reference type to the closure type.

§ 3.9. [lex.name]

Modify [lex.name] as follows:

2. The identifiers in Table 4 have a special meaning when appearing in a certain context. When referred to in the grammar, these identifiers are used explicitly rather than using the *identifier* grammar production. Unless otherwise specified, any ambiguity as to whether a given *identifier* has a special meaning is resolved to interpret the token as a regular *identifier*.

Table 4 — Identifiers with special meaning

final	import	post	replaceable_if_eligible
override	module	pre	trivially_relocatable_if_eligible

§ 3.10. [library.class.props]

Modify [library.class.props] as follows:

1. Unless explicitly stated otherwise, it is unspecified whether any class described in [support] through [exec] and [depr] is a trivially copyable class, a trivially relocatable class, a standard-layout class, or an implicit-lifetime class ([class.prop]).

~~2. Unless explicitly stated otherwise, it is unspecified whether any class for which trivial relocation (i.e., the effects of `trivially_relocate` (`{obj.lifetime}`)) would be semantically equivalent to move-construction of the destination object followed by destruction of the source object is a trivially relocatable class (`{class.prop}`).~~

~~3. Unless explicitly stated otherwise, it is unspecified whether a class `C` is a replaceable class (`{class.prop}`) if assigning an xvalue `a` of type `C` to an object `b` of type `C` is semantically equivalent to destroying `b` and then constructing from `a` in `b`'s place.~~

§ 3.11. [meta.type.synop]

Modify [meta.type.synop] as follows:

```

[...]
```

```

// \[meta.unary.prop\], type properties
template<class T> struct is_const;
template<class T> struct is_volatile;
template<class T> struct is_trivially_copyable;
template<class T> struct is_trivially_relocatable;
template<class T> struct is_replaceable;
template<class T> struct is_standard_layout;
[...]
```

```

template<class T>
    constexpr bool is_implicit_lifetime_v = is_implicit_lifetime<T>::value;
template<class T>
    constexpr bool is_replaceable_v = is_replaceable<T>::value;
template<class T>
    constexpr bool has_virtual_destructor_v = has_virtual_destructor<T>::value;
```

§ 3.12. [\[meta.unary.prop\]](#)

Modify Table 54 in [\[meta.unary.prop\]](#) as follows:

Template	Condition	Preconditions
template<class T> struct is_trivially_copyable;	T is a trivially copyable type ([basic.types.general])	remove_all_extents_t<T> shall be a complete type or cv void.
template<class T> struct is_trivially_relocatable;	T is a trivially relocatable type ([basic.types.general])	remove_all_extents_t<T> shall be a complete type or cv void.
template<class T> struct is_replaceable;	T is a replaceable type ([basic.types.general])	remove_all_extents_t<T> shall be a complete type or cv void.
template<class T> struct is_standard_layout;	T is a standard-layout type ([basic.types.general])	remove_all_extents_t<T> shall be a complete type or cv void.

§ 3.13. [\[obj.lifetime\]](#)

Modify [\[obj.lifetime\]](#) as follows:

```
template<class T>
    T* trivially_relocate(T* first, T* last, T* result);
```

9. *Mandates:* `is_trivially_relocatable_v<T> && !is_const_v<T>` is true. T is not an array of unknown bound.

10. *Preconditions:*

- `[first, last)` is a valid range.
- `[result, result + (last - first))` denotes a region of storage that is a subset of the region reachable through `result` ([`basic.compound`]) and suitably aligned for the type T.
- No element in the range `[first, last)` is a potentially-overlapping subobject.

11. *Postconditions:* No effect if `result == first` is true. Otherwise, the range denoted by `[result, result + (last - first))` contains objects (including subobjects) whose lifetime has begun and whose object representations are the original object representations of the corresponding objects in the source range `[first, last)` except for any parts of the object representations used by the implementation to represent type information (`[intro.object]`). If any of the objects has union type, its active member is the same as that of the corresponding object in the source range. If any of the aforementioned objects has a non-static data member of reference type, that reference refers to the same entity as does the corresponding reference in the source range. The lifetimes of the original objects in the source range have ended.

12. *Returns:* `result + (last - first)`.

13. *Throws:* Nothing.

14. *Complexity:* Linear in the length of the source range.

15. *Remarks:* The destination region of storage is considered reused ([`basic.life`]). No constructors or destructors are invoked.

[*Note:* Overlapping ranges are supported. —end note]

§ 3.14. [diff.cpp23.dcl.dcl]

Modify [diff.cpp23.dcl.dcl] as follows:

1. Affected subclause: `[dcl.decl.general]`

Change: Introduction of `trivially_relocatable_if_eligible` and `replaceable_if_eligible` as identifiers with special meaning (`[lex.name]`).

Rationale: Support declaration of trivially relocatable and replaceable types (`[class.prop]`).

Effect on original feature: Valid C++ 2023 code can become ill-formed.

Example:

```
struct C {};  
struct C replaceable_if_eligible {}; // was well-formed (new variable replaceable_if  
                                     // now ill-formed (redefines C)
```

—end example

§ References

§ Informative References

[N5008]

Working Draft — Programming Languages — C++. March 2025. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/n5008.pdf>

[P1144R10]

Arthur O'Dwyer. *std::is_trivially_relocatable*. February 2024. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p1144r10.html>

[P2786R13]

Pablo Halpern, Joshua Berne, Corentin Jabot, Pablo Halpern, Lori Hughes. *Trivial Relocatability For C++26*. 14 February 2025. URL: <https://wg21.link/p2786r13>

[P2786R4]

Mungo Gill, Alisdair Meredith. *Trivial Relocatability For C++26*. 9 February 2024. URL: <https://wg21.link/p2786r4>

[P2959]

Alisdair Meredith. *Relocation within containers*. October 2023. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2959r0.html>

[P2967]

Alisdair Meredith; Mungo Gill. *Relocation has a library interface*. May 2024. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p2967r1.pdf>

[P3236]

Alan de Freitas; Daniel Liam Anderson; Giuseppe D'Angelo; Hans Goudey; Jacques Lucke; Krystian Stasiowski; Stéphane Janel; Thiago Maciera. *Please reject P2786 and adopt P1144*.

May 2024. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3236r1.html>

[P3516]

Louis Dionne; Giuseppe D'Angelo. *Uninitialized algorithms for relocation*. February 2025. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3516r1.html>